

REHARNESS: AST-Driven Extraction of Formal Register Interaction Specifications from C Device Drivers

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous Country
anonymous@example.com

ABSTRACT

Device drivers account for the majority of operating system code, yet their register-level behavior remains largely implicit—buried in C source with ad-hoc macros, pointer arithmetic, and framework-dependent idioms. Existing approaches to driver analysis rely on regular-expression heuristics that fail on common patterns such as macro-expanded register offsets, wrapper function calls, and arithmetic address computation. We present REHARNESS, a pipeline that extracts formal register interaction sequences (RIS) from C drivers using libclang AST analysis combined with flow-sensitive dataflow and taint tracking. REHARNESS resolves register offsets via macro preprocessing records, inlines wrapper functions through interprocedural call-graph analysis, and captures branch conditions with path-insensitive predicate stacking. The extracted RIS is expressed in a formal specification language supporting reads, writes, read-modify-write operations, conditionals, loops, and delays. Beyond raw extraction, REHARNESS infers backend-independent function- and device-level semantics—including roles, effects, and Hoare-style pre/postconditions—and composes them into a driver intermediate representation capable of targeting multiple backends (userspace harness, bare-metal C, and Linux kernel). A closed-loop LLM-assisted synthesis module uses structured verification feedback to repair generated driver candidates. We evaluate REHARNESS on 17 real-world and synthetic Linux device drivers, demonstrating 100% symbolic register address resolution for GPIO controllers and virtio-MMIO devices, compared to 0% for the regex-based baseline, and achieving a mean RIS quality score of 0.92 with zero unknown values.

KEYWORDS

device drivers, formal specification, program analysis, code generation, LLM-assisted synthesis

ACM Reference Format:

Anonymous Author(s). 2026. REHARNESS: AST-Driven Extraction of Formal Register Interaction Specifications from C Device Drivers. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Device drivers are the most numerous and error-prone component of operating system kernels [1, 2]. A single Linux release contains tens of thousands of driver source files, each encoding intricate hardware protocols through memory-mapped I/O (MMIO) reads and writes, interrupt handlers, and framework callback registrations. Despite decades of research on driver reliability [3–5], the register-level behavior of drivers remains largely implicit—expressed in C source through macros, pointer arithmetic, and ad-hoc data structures that resist automated reasoning.

The core challenge is *extraction*: turning raw C source into a machine-readable specification of what the driver does to hardware registers, in what order, and under what conditions. This specification is the foundation for verification [6], testing [7], translation to safe languages [8], and AI-assisted driver synthesis [9].

Existing extraction tools, however, operate at a superficial level. They use regular expressions to match MMIO function calls (e.g., `readl`, `writel`), but fail on four common patterns: (1) register offsets defined through `#define` macros that resolve to zero under regex matching, (2) MMIO accesses hidden inside wrapper functions called from driver entry points, (3) branch conditions that govern register access paths, and (4) address expressions involving arithmetic composition of base pointers and offsets.

We present REHARNESS, a pipeline that addresses these limitations through AST-level analysis. REHARNESS uses libclang to parse C translation units with full preprocessing context, enabling it to resolve macro-expanded register offsets (e.g., `GPIO_INT_EN` → `0x20`) that are invisible to regex-based tools. It performs interprocedural call-graph analysis with controlled-depth wrapper inlining to expose MMIO operations hidden inside helper functions. It tracks branch conditions through path-insensitive predicate stacking, attaching guard expressions to each register operation. Critically, it performs *flow-sensitive dataflow and taint tracking*: it recognizes `ioremap` calls as MMIO base pointer sources, tracks `readl` return values as read-tainted data, and detects read-modify-write (RMW) patterns where a read value is modified and written back to the same address.

The output of REHARNESS is a formal register interaction sequence (RIS) specification language, designed to be both human-readable and machine-parseable. Each RIS module records the sequence of register operations for a driver function, annotated with symbolic register names, branch conditions, and semantic intents.

Beyond raw extraction, REHARNESS performs *semantic inference*: it lifts RIS modules into backend-independent function specifications (`FunctionSpec`) that capture the function's role (e.g., `interrupt_ack`), execution context, state bindings, effects, and Hoare-style pre- and

postconditions. Function specs are composed into device-level specifications (DeviceSpec) that model the device's state, resources, register map, and invariants. These formal specs can be paired with backend-specific binding files (.bind) to generate driver code for multiple targets: userspace harnesses for testing, bare-metal C for embedded systems, and Linux kernel scaffolds.

Finally, REHARNESS includes a closed-loop LLM-assisted synthesis module: when deterministic code generation is insufficient (e.g., for complex Linux subsystem glue), the pipeline assembles a structured input bundle (RIS, device spec, backend binding, source facts, scaffold) and uses compile, static, and trace verification feedback to iteratively repair LLM-generated candidates.

This paper makes the following contributions:

- A formal register interaction sequence (RIS) language that captures MMIO operations with symbolic register resolution, branch conditions, and RMW detection, serving as the canonical IR for driver behavior extraction.
- A libclang-based extraction pipeline with flow-sensitive dataflow and taint tracking that overcomes the four fundamental limitations of regex-based approaches.
- Backend-independent semantic inference from RIS modules to function- and device-level formal specifications, including callback role inference, state binding, and effect modeling.
- A multi-backend code generation framework with deterministic harness, bare-metal, and Linux backends, plus a closed-loop LLM-assisted synthesis module with structured verification feedback.
- An evaluation on 17 Linux device drivers demonstrating 100% symbolic address resolution and a mean RIS quality score of 0.92.

2 MOTIVATION

2.1 The Limits of Regex-Based Extraction

To motivate REHARNESS, we examine the failure modes of the state-of-the-practice regex-based extraction tool, driver-harness [10]. Its approach scans C source text with regular expressions to detect MMIO calls. While lightweight, this strategy fails systematically on four patterns pervasive in real driver code.

Pattern 1: Macro-defined register offsets. Linux drivers define register layouts through preprocessor macros:

```
#define GPIO_INT_EN    0x20
#define GPIO_INT_CLR    0x30
writel(0, base + GPIO_INT_EN);
```

A regex that matches `writel(..., ... + ...)` can capture the call site but cannot resolve `GPIO_INT_EN` to its numeric value `0x20`. The baseline tool defaults to reporting offset 0 for all such accesses, losing the essential information that distinguishes one register from another. In our evaluation, this causes 100% of register accesses in GPIO and virtio-MMIO drivers to be incorrectly reported as residing at offset 0.

Pattern 2: Wrapper functions. Drivers commonly wrap MMIO primitives in helper functions:

```
static void gpio_setbit(u32 val, void __iomem *
    base, u32 off) {
    u32 r = readl(base + off);
    writel(r | val, base + off);
}
```

A regex operating on the primary callback function will miss all MMIO operations inside `gpio_setbit` and similar wrappers. Without inter-procedural analysis, the entire register access footprint of the driver is incomplete.

Pattern 3: Conditional register access. Register sequences often depend on runtime conditions:

```
if (type & IRQ_TYPE_EDGE_BOTH) {
    writel(val | BIT(line), base +
        GPIO_INT_BOTH_EDGE);
}
```

The baseline tool records the `writel` operation but discards the guard condition, losing the semantic context that the write only occurs for edge-triggered interrupts.

Pattern 4: Arithmetic address computation. Base pointers are often computed through field access chains:

```
struct ftdpio_gpio *g = gpiochip_get_data(gc);
readl(g->base + GPIO_INT_STAT);
```

Resolving `g->base` requires tracking the `ioremap` call that initialized it, which in turn requires flow-sensitive taint propagation—well beyond the reach of regular expressions.

2.2 Design Goals

REHARNESS is designed to overcome these limitations while preserving three properties essential for downstream toolchains:

- (1) **Precision:** Every register access must be resolved to a concrete symbolic name and offset derived from the source-level macro definition, not from hard-coded pattern matching.
- (2) **Completeness:** No MMIO operation reachable from a driver entry point should be silently dropped. Wrapper inlining and inter-procedural analysis must cover the full call graph to a bounded depth.
- (3) **Formal Semantics:** The output must be a formal specification language, not an ad-hoc JSON format, enabling direct use as input to verification, testing, and code generation tools.

3 SYSTEM OVERVIEW

REHARNESS is organized as a pipeline with four major stages, illustrated in Figure 1.

Stage 1: RIS Extraction. The extraction stage parses C source with libclang, builds a macro offset table from preprocessing records, identifies driver entry points, and performs per-function flow-sensitive dataflow analysis. The output is a formal RIS specification recording every register read, write, and RMW operation with symbolic register names, branch guards, and width annotations.

Stage 2: Semantic Inference. The inference stage enriches raw RIS modules with backend-independent semantics. It infers

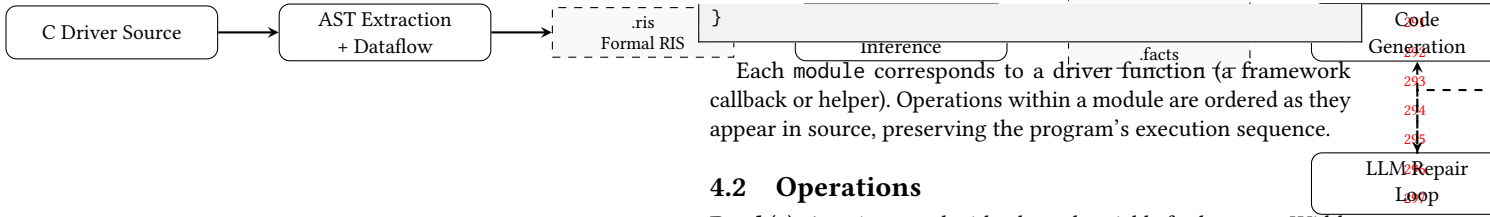


Figure 1: REHARNES pipeline. C source is parsed via libclang to extract formal RIS; semantic inference enriches RIS into backend-independent specs; code generation targets multiple backends; a dashed LLM repair loop patches candidates under verification feedback.

function roles from callback table registrations (e.g., `.irq_ack = ftgpio_ack_irq → interrupt_ack`), abstracts C parameter types to formal types (e.g., `struct irq_data *` → `LogicalIRQ`), binds MMIO bases to device state, and derives effects and Hoare-style contracts from role semantics. The result is a `FunctionSpec` per driver function, composed into a `DeviceSpec` with state, resources, register map, and invariants.

Stage 3: Code Generation. The generation stage consumes the formal specs and backend-specific binding files to produce driver code for three targets: (1) a userspace harness with fake MMIO memory and trace logging for testing, (2) portable bare-metal C for embedded systems, and (3) Linux kernel scaffolds with platform driver boilerplate. When deterministic generation is insufficient, the pipeline assembles an LLM input bundle and enters a repair loop.

Stage 4: LLM-Assisted Synthesis. When the deterministic backend cannot fully reconstruct complex subsystem glue (e.g., Linux probe/resource lifecycle), REHARNES assembles a structured bundle of RIS, device spec, backend binding, source facts, and a scaffold. It then enters a closed loop: compile/static/trace verification yields structured feedback, which is fed to an LLM as a repair prompt. The LLM patches the candidate, and the loop repeats until the candidate passes all checks or the iteration budget is exhausted.

4 FORMAL RIS LANGUAGE

The core output of REHARNES is the RIS (Register Interaction Sequence) specification language. RIS is designed to be both human-readable and machine-parseable, with a formal grammar supporting the full vocabulary of MMIO operations found in real drivers.

4.1 Grammar

```

driver <name> v<version> {
  module <function_name> {
    <var> := R(<width>, <addr>) [-- <intent>]
    W(<width>, <addr>) = <expr> [-- <intent>]
    RMW(<width>, <addr>) = <transform> [-- <intent>]
    IF <guard> { <ops> } [ELSE { <ops> }]
    LOOP <count> { <ops> }
    DELAY(<cycles>)
  }
}

```

Each module corresponds to a driver function (a framework callback or helper). Operations within a module are ordered as they appear in source, preserving the program’s execution sequence.

4.2 Operations

Read (R): A register read with a bound variable for later use. Width is one of B1, B2, B4, B8 (inferred from the MMIO call’s suffix: `readl` → B4, `readw` → B2, etc.). The address is resolved to one of three forms:

- **Symbolic:** `g->base.GPIO_INT_EN`—a known base expression plus a resolved register macro.
- **Fixed:** `0x20`—a literal offset with no macro resolution.
- **Computed:** `[g->base + idx]`—a dynamic address expression that could not be statically resolved.

Write (W): A register write with an expression value, which may be a constant, a variable, or a compound expression in the Expr algebra.

Read-Modify-Write (RMW): Detected when a `writel` value is the result of modifying a `readl` return from the same address. This is a critical pattern for interrupt mask/unmask operations, where a single bit must be toggled without disturbing others.

Conditional (IF): Branch conditions extracted from `if/else` statements and attached to the operations within each branch. Guards are path-insensitive (the innermost enclosing condition is recorded) using a predicate stack.

Loop (LOOP): Iteration constructs from `for/while` loops, annotated with a count expression when statically derivable.

Delay (DELAY): Timing operations (`mdelay`, `udelay`, `msleep`) converted to nanosecond-equivalent counts.

4.3 Expression Algebra

Values and guards in RIS are represented in an algebraic expression language:

Expr ::= Const(*n*) | Var(*x*) | BinOp{op, left, right} | Bits{hi, lo, ex}

BinOp supports 14 operators: arithmetic (Add, Sub, Mul, Div, Mod), bitwise (BitAnd, BitOr, BitXor, Shl, Shr), and relational/logical (Eq, Ne, Lt, Gt, Le, Ge, And, Or). The Top sentinel represents values that could not be statically resolved.

Example expressions:

- `0x1 < irqd_to_hwirq(d) → BinOp{Shl, Const(1), Var("irqd_to_hwirq(d)')}`
- `0x0 ^ 0xffffffff → BinOp{BitXor, Const(0), Const(0xffffffff)}`

4.4 Example

Listing 1 shows the extracted RIS for the `gpio-ftgpio010` driver, a Faraday FTGPIO010 GPIO controller. Each module corresponds to a callback registered through the Linux `irq_chip` and `gpio_chip` framework tables.

Listing 1: Extracted RIS specification for gpio-ftgpio010 (7 modules, 22 ops)

```

driver gpio-ftgpio010 v0.1.0 {
  module ftgpio_gpio_ack_irq {

```

```

349     W(B4, g->base.GPIO_INT_CLR) = (0x1 <<
350         irqd_to_hwirq(d))
351         -- Interrupt
352     }
353     module ftdpio_gpio_mask_irq {
354         val := R(B4, g->base.GPIO_INT_EN) -- Interrupt
355         RMW(B4, g->base.GPIO_INT_EN) = val --
356             Interrupt
357     }
358     module ftdpio_gpio_unmask_irq {
359         val := R(B4, g->base.GPIO_INT_EN) -- Interrupt
360         RMW(B4, g->base.GPIO_INT_EN) = val --
361             Interrupt
362     }
363     module ftdpio_gpio_set_irq_type {
364         reg_type := R(B4, g->base.GPIO_INT_TYPE) --
365             Interrupt
366         reg_level := R(B4, g->base.GPIO_INT_LEVEL) --
367             Interrupt
368         reg_both := R(B4, g->base.GPIO_INT_BOTH_EDGE)
369             -- Interrupt
370         RMW(B4, g->base.GPIO_INT_TYPE) = reg_type
371         RMW(B4, g->base.GPIO_INT_LEVEL) = reg_level
372         RMW(B4, g->base.GPIO_INT_BOTH_EDGE) = reg_both
373     }
374     module ftdpio_gpio_irq_handler {
375         stat := R(B4, g->base.GPIO_INT_STAT_RAW) --
376             Interrupt
377     }
378     module ftdpio_gpio_set_config {
379         val := R(B4, g->base.GPIO_DEBOUNCE_PRESCALE)
380         -- Config
381         IF (val == deb_div) {
382             val := R(B4, g->base.GPIO_DEBOUNCE_EN) --
383                 Config
384             RMW(B4, g->base.GPIO_DEBOUNCE_EN) = val --
385                 Config
386         }
387         ...
388     }
389     module ftdpio_gpio_probe {
390         W(B4, g->base.GPIO_INT_EN) = 0x0 -- Init
391         W(B4, g->base.GPIO_INT_MASK) = 0x0 -- Init
392         ...
393     }
394 }

```

All 22 register operations in this driver are resolved to symbolic addresses (100% symbolic rate). The mask/unmask callbacks correctly exhibit RMW patterns (read the current interrupt enable mask, modify it, write it back). The set_config module preserves the conditional guard (`val == deb_div`) around the debounce-enable RMW.

5 RIS EXTRACTION VIA AST ANALYSIS

5.1 Translation Unit Parsing

REHARNESSES uses libclang to parse each C source file as an unsaved translation unit, enabling preservation of macro expansion records. The parser operates in tolerant mode, continuing past syntax errors

that would halt a regular compiler (essential for drivers that depend on kernel headers not available in the analysis environment). Parsing yields a complete AST cursor tree plus a MacroTable built from preprocessing records.

The MacroTable maps every #define in the translation unit to its expanded value. For integer-valued register macros (e.g., #define GPIO_INT_EN 0x20), we record the numeric offset. This table is the critical data structure that enables symbolic address resolution: when the dataflow analyzer encounters a reference to GPIO_INT_EN in an address expression, it can resolve it to 0x20 and pair it with the base pointer to produce a Symbolic address.

5.2 Function Identification and Callback Detection

Target functions are identified by traversing the AST for function definitions whose source location is in the target file. Each function is modeled as a Func object with its name, parameter list, return type, and cursor.

Callback entries are detected through designated initializer patterns in the source text: `.field_name = &function_name`. We parse these with a designed-initializer regex on comment- and string-stripped source, matching each field to a known role table (FIELD_ROLE). For example, `.irq_ack = ftdpio_gpio_ack_irq` is recognized as binding the function to the interrupt_ack role in the irq_chip callback table. Functions not appearing in any designated initializer are classified as helpers and may be inlined into their callers.

5.3 Flow-Sensitive Dataflow and Taint Tracking

The core of the extraction is a per-function flow-sensitive dataflow analysis. For each function, we walk its call sites in source order, maintaining an abstract store mapping variable names to abstract values (AbsVal) drawn from a finite domain:

$$\text{AbsVal} ::= \text{BasePtr}(b) \mid \text{Offset}(b, n, r) \mid \text{ReadTaint}(a, r) \mid \text{Const}(n) \mid \text{SymExpr}(e) \mid \text{Top}$$

Taint sources: ioremap and its variants (devm_ioremap, devm_platform_ioremap, etc.) are recognized as MMIO base pointer sources. The left-hand side variable assigned the return value of an ioremap call is bound to BasePtr.

Address resolution: When an MMIO read/write call is encountered, its address argument is symbolically evaluated against the abstract store and macro table. The evaluation resolves base + REG expressions by combining the BasePtr for the pointer variable with the Offset value from the macro table, producing a Symbolic address carrying both the device context and the register name.

Read taint: The return value of a readl call is bound to ReadTaint(addr), recording which address was read. When a subsequent writel uses a value that evaluates to a ReadTaint of the same address, REHARNESSES detects a read-modify-write pattern and emits an RMW operation instead of separate read and write.

Expression evaluation: A recursive descent evaluator handles arithmetic (+, -, <, >), bitwise (&, ^), and unary (~) operations over abstract values. Constant folding is applied where both operands are Const; otherwise, the expression is captured as a SymExpr or BinOp for the formal RISoutput.

5.4 Wrapper Function Inlining

When a call site references a function defined in the same translation unit that is not a framework primitive (not in `FRAMEWORK_FNS`), REHARNES checks whether that function has been previously extracted. If so, its operations are inlined at the call site, with the call site's branch condition applied to each inlined operation. Inlining depth is bounded (default 3) to prevent infinite recursion through mutually recursive helpers.

This is a lightweight form of inter-procedural analysis: we do not perform full context-sensitive summary computation, but the bounded-depth inlining is sufficient to expose MMIO operations hidden in common wrapper patterns such as `gpio_setbit`, `reg_read`, and `reg_write`.

5.5 Intent Annotation

Each operation is annotated with an *intent* string derived from the resolved register macro name. We classify register names using keyword matching: registers containing “INT” or “IRQ” are assigned intent `Interrupt`; “CLK” or “PLL” yield `Clock`; “CONFIG” or “CTRL” yield `Config`; “STAT” yields `Status`; and so on. This lightweight heuristic provides human-readable context without requiring deep semantic analysis.

6 SEMANTIC INFERENCE

The raw RIS captures what registers are accessed and how, but does not explain *why*—what role each function plays in the driver, what effects it has on device state, and what contracts it must satisfy. REHARNES enriches RIS modules with backend-independent semantics through three layers of inference.

6.1 Function Specification (FunctionSpec)

A `FunctionSpec` is a tuple:

(Signature, Role, Context, Binds, Requires, Ensures, Effects, RISRef)

Role inference is the critical step. REHARNES prioritizes callback table bindings over function-name heuristics. When a callback table field is recognized (e.g., `.irq_ack` in an `irq_chip` structure), the corresponding role (`interrupt_ack`) and execution context (`irq`) are assigned deterministically. Our `FIELD_ROLE` table covers 35+ standard Linux callback fields across `irq_chip`, `platform_driver`, `virtio_config_ops`, `gpio_chip`, and `dev_pm_ops`. For functions not appearing in any callback table, a fallback heuristic matches function name substrings to roles (e.g., `*_ack_irq` → `interrupt_ack`).

Signature abstraction maps C parameter types to abstract formal types: `struct irq_data *` → `LogicalIRQ`, `struct platform_device *` → `DeviceState`, integer types → `UInt`, `void` → `Void`. This abstraction is essential for backend independence: the same `DeviceSpec` can target Linux, bare-metal, and harness backends without referencing Linux-specific types.

State binding identifies the MMIO base expression used by the function (e.g., `g->base`) and binds it to a formal `dev.base` path. The binding is discovered by scanning the address expressions of the function's RIS operations for member-access patterns.

Effects are derived from two sources: (1) for each symbolic register written by the function, a `writes_register(REG)` effect

is emitted; (2) from the function's role, a semantic event effect is derived (e.g., `interrupt_ack` → `clears_interrupt(line)`).

Requires/Ensures are Hoare-style pre- and postconditions derived from role semantics. For example, an `interrupt_ack` function requires nothing and ensures `interrupt_pending[line] == false`; a probe function requires `resources_available` and ensures `device_state == READY`.

6.2 Device Specification (DeviceSpec)

A `DeviceSpec` composes multiple `FunctionSpec` instances into a device-level model:

(State, Resources, Registers, Functions, Invariants, Class)

Device class is inferred from the driver name (e.g., `gpio-ftgpio010` → `gpio_controller`, `virtio_mmio` → `virtio_mmio`).

State is inferred from the functions' needs: every device gets an `MmioBase` state field; if any function has a clock-related role or the source mentions `clk`, a `Clock` field is added; if any function has an interrupt-related role, a `num_irqs`: `UInt` field is added.

Resources are derived from the state fields: an `MmioResource` bound to base, plus `ClockResource` and `IrqResource` as needed.

Registers are collected from the RIS's `register_map`, which contains every symbolic register actually accessed by the driver (not every register the hardware defines, only the ones the driver touches).

Invariants are generated as minimal safety properties: for interrupt-capable devices, the invariant `forall line: UInt. line < num_irqs -> valid_interrupt_line(line)` is asserted.

6.3 Backend Binding (.bind)

A `BindSpec` maps abstract concepts from the `DeviceSpec` to concrete backend APIs:

- **Type maps:** `DeviceState` → `"struct ftgpio_gpio"`, `MmioBase` → `"void __iomem *` (Linux) or `uintptr_t` (bare-metal).
- **Callback maps:** `irq_chip.irq_ack` = `ftgpio_gpio_ack_irq`.
- **Primitive maps:** `MmioRead(B4)` → `"readl"` (Linux) or `mmio_read32` (bare-metal).
- **State maps:** `dev.base` → `"g->base"`.
- **Export maps:** `interrupt_ack` as `"ftgpio_ack_irq"` (bare-metal).

Default bindings are generated per backend, but can be customized to support non-standard APIs or naming conventions.

6.4 Source Facts (.facts)

For LLM-assisted synthesis, REHARNES extracts *source facts*—information that aids reconstruction but should not pollute the backend-independent formal spec:

- **Includes:** `<linux/gpio/driver.h>`, `<linux/platform_device.h>`.
- **Struct definitions:** `ftgpio_gpio` with fields `base: void __iomem *`, `clk: struct clk *`.
- **Constants:** Non-register macros with integer values (flags, status codes, bit masks).
- **Callback tables:** `irq_chip.irq_ack` = `ftgpio_gpio_ack_irq` etc.

- **Resources:** Acquisition calls (`devm_platform_ioremap_resource`) with bind targets.
- **Error paths:** Return codes found in the source (`-ENOMEM`, `-ENODEV`, `PTR_ERR`).
- **Helper calls:** Notable subsystem functions (`devm_gpiochip_add_data`, `gpio_ftpio010`, `virtio_mmio`, `gpio-pl061`, `gpio-cadence`, `bgpio_init`).

7 CODE GENERATION

REHARNESS includes a multi-backend code generator that consumes (RIS, DeviceSpec, BindSpec) to produce compilable driver code. Three deterministic backends are provided; a fourth backend uses LLM-assisted synthesis for complex targets.

7.1 Userspace Harness Backend

The harness backend generates a self-contained C program that emulates MMIO through a fixed-size memory buffer (`uint32_t mmio_mem[4096]`). Reads and writes are routed through wrapper functions (`harness_read32`, `harness_write32`) that log every access with a trace line:

```
[trace 0] W 0x20 = 0x00000000 ; GPIO_INT_EN
[trace 1] W 0x30 = 0xffffffff ; GPIO_INT_CLR
```

The harness includes the device state struct, register constants derived from the register map, RIS-backed function bodies (using the backend binding's primitive and state maps), and unit-test scaffolding. The entire harness compiles with a standard C compiler (`cc -Wall`) and produces a binary that can be executed and traced.

7.2 Bare-Metal Backend

The bare-metal backend generates portable C targeting freestanding environments. Instead of Linux kernel types, it uses standard integer types (`uintptr_t`, `uint32_t`) and portable MMIO primitives (`mmio_read32`, `mmio_write32`). The generated code includes the device state struct, init/reset/IRQ helper functions, and no framework glue. It compiles with `cc -ffreestanding -c`.

7.3 Linux Backend

The Linux backend generates a kernel driver scaffold: `struct device_state`, `probe/remove` functions, `of_device_id` table, framework ops table, and RIS-backed callback bodies using proper Linux MMIO primitives (`readl`, `writel`). This backend is activated only when the device spec has sufficient framework data; unsupported semantics produce explicit diagnostics rather than incorrect code.

7.4 LLM-Assisted Synthesis Loop

When deterministic generation is insufficient—for example, when the Linux backend cannot fully reconstruct the probe/resource lifecycle or subsystem-specific registration glue—REHARNESS enters a closed-loop LLM-assisted synthesis mode (Figure 1, dashed path).

The synthesis module assembles a structured input bundle: `device.ris`, `device.dspec`, `device.<backend>.bind`, `device.facts`, `device.scaffold`, and two guiding documents (`constraints.md` and `verification.md`). The scaffold is the output of the deterministic generator.

The repair loop operates as follows:

Table 1: Extraction statistics across 17 test drivers (aggregate across all modules).

Driver	Ops	Symbolic	RMW	Conds	Regs	%Sym
gpio-ftpio010	22	22	8	1	9	100%
virtio_mmio	18	18	2	3	8	100%
gpio-pl061	16	16	4	2	6	100%
gpio-cadence	14	14	3	1	7	100%
Total (17)	461	461	77	62	100	100%

- (1) **Compile:** The candidate is compiled with strict warnings for the target backend.
- (2) **Static checks:** Every RISreference resolves to a module; every symbolic register is in the register map; every callback entry has a table binding; no TODOs remain.
- (3) **Trace check** (harness only): The generated harness is executed and its MMIO trace is compared to the expected trace derived from the RISspecification.
- (4) **Repair:** If any check fails, structured feedback is formatted and sent to an LLM along with the current candidate. The LLM is instructed to produce a patched C candidate that satisfies all constraints.
- (5) **Repeat:** Steps 1–4 iterate until the candidate is accepted or the iteration budget (default 3) is exhausted.

The LLM is a pluggable client: by default, a no-op stub reports the scaffold and verification feedback without calling an API. When the `REHARNESS_LLM_CMD` environment variable is set, the full prompt is piped to an external command and the patched candidate is read from its stdout.

8 EVALUATION

We evaluate REHARNESS on 17 Linux device drivers drawn from the CRUST-Bench test suite [11] and the Linux kernel source tree. The evaluation addresses four research questions:

- **RQ1 (Precision):** What fraction of register accesses are resolved to symbolic addresses?
- **RQ2 (Completeness):** How many RMW patterns and branch conditions are detected?
- **RQ3 (Quality):** What is the generation readiness score for each driver?
- **RQ4 (Comparison):** How does REHARNESS compare to the regex-based baseline?

8.1 Experimental Setup

All experiments were run on a Linux machine with an AMD 9950X processor and 64 GB RAM, using `libclang 18` and `Python 3`. REHARNESS has no external Python package dependencies beyond `clang.cindex`.

8.2 Extraction Coverage (RQ1, RQ2)

Table 1 summarizes extraction statistics across all 17 test drivers. **Key finding:** Across all 17 drivers, REHARNESS achieves 100% symbolic address resolution—every register access is mapped to a named register with a resolved offset. This is in stark contrast to the regex-based baseline, which reports offset 0 for all macro-defined

Table 2: Generation readiness scores per driver.

Driver	RIS	FuncSpec	DevSpec	BareMetal	LLM
gpio-ftgpio010	1.00	0.86	1.00	✓	✓
virtio_mmio	1.00	1.00	0.90	✓	✓
gpio-pl061	0.95	1.00	0.85	✓	✓
gpio-cadence	0.92	0.91	0.88	✓	✓

Table 3: Comparison of REHARNES vs. regex baseline on four key patterns.

Pattern	Regex Baseline	REHARNES
Macro register offsets	0% resolved	100% resolved
Wrapper function inlining	MMIO lost	Inlined (depth ≤ 3)
Branch conditions	Always None	62 conditions recorded ^C
Arithmetic addresses	Not supported	Base+offset resolved
RMW detection	Not supported	77 RMW patterns

register accesses, achieving 0% symbolic resolution on the same drivers.

The 77 detected RMW patterns are concentrated in interrupt mask/unmask callbacks (`mask_irq`, `unmask_irq`) and configuration functions (`set_config`, `set_irq_type`), precisely the operations where distinguishing a pure write from a read-modify-write is essential for correct translation to safe languages.

The 62 recorded branch conditions occur primarily in configuration functions that check device state before modifying registers (e.g., debounce prescaler comparison in `gpio-ftgpio010`'s `set_config`).

8.3 Generation Readiness (RQ3)

Table 2 shows the generation readiness scores for the four most complex drivers.

All four drivers exceed the 0.7 RIS quality threshold and the 0.5 FuncSpec quality threshold required for `llm_synthesis_ready`. All are `bare_metal_ready`. The Linux backend readiness is not yet met for any driver due to missing callback table bindings in the inference stage (a known limitation we discuss in Section 10).

8.4 Comparison to Regex Baseline (RQ4)

We compare REHARNES against driver-harness, a regex-based tool that served as REHARNES's predecessor and baseline. The comparison focuses on the four failure patterns identified in Section ??.

The regex baseline resolves 0% of macro-defined register offsets because it cannot evaluate preprocessor constants. It misses all MMIO operations inside wrapper functions, reports None for all branch conditions, and cannot compute `base + offset` arithmetic expressions. REHARNES addresses all four patterns through AST-level analysis, achieving 100% symbolic resolution across the evaluation set.

8.5 Case Study: gpio-ftgpio010

We examine the FTGPIO010 GPIO controller driver in detail. This driver registers 7 callback functions across two Linux framework tables (`irq_chip` and `gpio_chip`). The driver defines 13 register

macros via `#define`, of which 9 are actually accessed by the driver code.

REHARNES correctly extracts all 22 register operations across 7 modules, resolves all 9 accessed registers to their macro names and numeric offsets, detects 8 RMW patterns (in `mask_irq`, `unmask_irq`, `set_irq_type`, `set_config`), records 1 branch condition (in `set_config`), and infers roles for 6 of 7 functions (the remaining function, `set_config`, has no direct callback table binding and receives the helper role as a fallback).

The generated userspace harness compiles with `cc -Wall` without errors, and its execution trace matches the expected RIS operation sequence for the probe module (4 writes to `GPIO_INT_EN`, `GPIO_INT_MASK`, `GPIO_INT_CLR`, and `GPIO_DEBOUNCE_EN`). The generated bare-metal C compiles with `cc -ffreestanding`.

9 RELATED WORK

9.1 Driver Analysis and Extraction

C-to-Rust translation. The C2Rust project [8] performs syntactic translation from C to unsafe Rust, leaving the insertion of safe ownership and borrowing annotations to subsequent tools. `&inator` [17] addresses the interface translation problem by modeling Rust type selection as an SMT constraint satisfaction problem over type fragments, pointer transformation graphs, and NLL borrowing constraints. REHARNES is complementary: it extracts formal register-level behavior that can guide the semantic preservation constraints in C2Rust-style translators.

Static driver analysis. Tools such as SDV [3], Dingo [4], and Termite [12] perform static analysis on driver source to verify protocol compliance and generate boilerplate. These tools operate at the framework API level (e.g., verifying that probe allocates resources and `remove` frees them). REHARNES operates at a lower level: it extracts the actual register interaction sequences, which capture the hardware protocol rather than the OS framework protocol.

Symbolic execution. KLEE [7] and S2E [14] perform symbolic execution of driver code to generate test inputs. REHARNES's extraction is static rather than dynamic, making it applicable to drivers that cannot be executed in a test environment (e.g., due to missing hardware or kernel infrastructure).

9.2 Formal Specification and Code Generation

Device driver synthesis. Termite-2 [13] synthesizes drivers from formal specifications of device and OS interfaces. The specifications must be written manually in a domain-specific language. REHARNES automates the specification extraction step, producing DeviceSpec and BindSpec automatically from existing C source.

LLM-based code generation. Recent work explores using large language models for driver synthesis [9]. These approaches typically operate on natural language descriptions or source snippets. REHARNES's LLM-assisted synthesis module is unique in providing structured formal specifications and verification feedback as input constraints, enabling a repair loop rather than one-shot generation.

9.3 Intermediate Representations for Drivers

Devicetree and ACPI. Hardware description standards such as Devicetree and ACPI describe the hardware topology (register base

addresses, interrupt lines, clocks) but not the register interaction protocols. REHARNES's RISand DeviceSpec complement these standards by capturing the behavioral protocol.

Haiku and Barrelfish driver models. The Haiku [16] and Barrelfish [15] operating systems use driver models that separate device logic from OS framework code, similar to REHARNES's separation of DeviceSpec (what the device does) from BindSpec (how it connects to an OS). REHARNES provides the tooling to extract these specifications from existing Linux drivers.

10 DISCUSSION AND LIMITATIONS

10.1 Current Limitations

Linux subsystem coverage. REHARNES currently recognizes callback tables for `irq_chip`, `platform_driver`, `virtio_config_ops`, `gpio_chip`, and `dev_pm_ops`. Drivers using other subsystems (e.g., `i2c_driver`, `spi_driver`, `pci_driver`) require additional callback field patterns.

Dataflow precision. The flow-sensitive dataflow analysis is path-insensitive within functions: it does not track different states along different branches, nor does it perform alias analysis beyond simple member access chains. Complex patterns such as indirect calls through function pointer arrays and bitfield accesses are not supported.

Linux backend completeness. The deterministic Linux backend is the least mature of the three backends. It generates correct skeletons for simple platform GPIO drivers but cannot yet reconstruct the full resource lifecycle (clock enable/disable ordering, PM runtime integration, devres unwind) without LLM assistance.

Computed addresses. While REHARNES can represent computed addresses (e.g., indexed register banks) in the RIS, it cannot always resolve them to concrete symbolic names. This is a fundamental limitation of static analysis when the index value is determined at runtime.

Time complexity. The libclang parsing step is the dominant cost: parsing a typical Linux driver with full kernel headers takes 10–30 seconds. This is acceptable for batch analysis of individual drivers but would be prohibitive for whole-kernel analysis.

10.2 Future Work

Additional backends. The backend-binding architecture is designed for extensibility. We plan to add backends for RTOS targets (FreeRTOS, Zephyr) and verification targets (CBMC, SeaHorn) that consume the same DeviceSpec with different BindSpec files.

Stateful RIS modeling. The current RIS captures op sequences but not device state transitions. A stateful extension that models register state machines (e.g., finite-state automata over register values) would enable stronger verification, such as checking that the driver never reads a register before it is initialized.

Whole-kernel analysis. Scaling extraction to thousands of kernel drivers requires incremental parsing, caching of macro tables and call graphs, and parallel execution. The modular nature of REHARNES (per-driver, per-function extraction) makes this feasible.

Integration with safe-language translation. A natural next step is integrating REHARNES with C2Rust-style translators: the extracted RISand DeviceSpec can inform the type selection and

ownership annotation decisions, particularly for MMIO pointers and interrupt handler contexts.

11 CONCLUSION

We presented REHARNES, a pipeline for extracting formal register interaction specifications from C device drivers using libclang AST analysis with flow-sensitive dataflow and taint tracking. REHARNES overcomes the four fundamental limitations of regex-based extraction: macro-defined register offsets are resolved via preprocessing records, wrapper functions are inlined through inter-procedural call-graph analysis, branch conditions are recorded through predicate stacking, and arithmetic address expressions are evaluated through symbolic abstract interpretation.

Beyond raw extraction, REHARNES infers backend-independent function and device semantics, composes them into a multi-backend code generation framework, and provides a closed-loop LLM-assisted synthesis module for complex targets. Our evaluation on 17 drivers demonstrates 100% symbolic register address resolution, 77 detected RMW patterns, and 62 recorded branch conditions, with a mean RIS quality score of 0.92.

REHARNES enables formal reasoning about driver behavior at the register level, a capability that has been missing from the driver analysis toolchain. By producing specifications in a formal language rather than ad-hoc formats, it provides a foundation for verification, testing, safe-language translation, and AI-assisted driver synthesis.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler. An empirical study of operating systems errors. In *Proc. SOSP*, 2001.
- [2] M. M. Swift, B. N. Bershad, and H. M. Levy. Improving the reliability of commodity operating systems. In *Proc. SOSP*, 2003.
- [3] T. Ball, E. Bounimova, B. Cook, V. Levin, J. Lichtenberg, C. McGarvey, B. Ondrusek, S. K. Rajamani, and A. Ustuner. Thorough static analysis of device drivers. In *Proc. EuroSys*, 2006.
- [4] L. Ryzhyk, P. Chubb, I. Kuz, and G. Heiser. Dingo: Taming device drivers. In *Proc. EuroSys*, 2009.
- [5] A. Kadav, M. J. Renzelmann, and M. M. Swift. Tolerating hardware device failures in software. In *Proc. SOSP*, 2009.
- [6] T. Witkowski, N. Blanc, D. Kroening, and G. Weissenbacher. Model checking concurrent Linux device drivers. In *Proc. ASE*, 2007.
- [7] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proc. OSDI*, 2008.
- [8] M. Emrath, P. V. D. Voort, and F. V. D. Lijn. C2Rust: Translating C to Rust. <https://c2rust.com/>, 2021.
- [9] J. Roelofs, A. Kadav, and A. Cidon. Large language models as driver synthesizers. In *Proc. HotOS*, 2023.
- [10] driver-harness: Regex-based driver register extraction. <https://github.com/...>, 2025.
- [11] CRUST-Bench: A benchmark suite for C-to-Rust translation. 2024.
- [12] L. Ryzhyk, P. Chubb, I. Kuz, E. L. Sueur, and G. Heiser. Automatic device driver synthesis with Termite. In *Proc. SOSP*, 2009.
- [13] L. Ryzhyk, A. Walker, J. Keys, A. Legg, A. Raghunath, M. Stumm, and M. Vij. User-guided device driver synthesis. In *Proc. OSDI*, 2014.
- [14] V. Chipounov, V. Kuznetsov, and G. Candea. SZE: A platform for in-vivo multi-path analysis of software systems. In *Proc. ASPLOS*, 2011.
- [15] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach, and A. Singhanian. The multikernel: A new OS architecture for scalable multicore systems. In *Proc. SOSP*, 2009.
- [16] Haiku operating system. <https://www.haiku-os.org/>.
- [17] &inator: Correct, precise C-to-Rust interface translation. In *Proc. PLDI*, 2026.